Laboratoire de tests, incidents et preuves — AAM O94.5

Dossier de validation AAM

20 octobre 2026

Table des matières

1 Description **2**

2 Architecture et protocole expérimental **2**

3 Phase P0 — Audit et reproduction du défaut **2**
3.1 Problèmes observés . . . 2
3.2 Test de référence . . . 2

4 Phase P1 — Modèles Transformers et poids locaux **2**
4.1 Architecture . . . 2
4.2 Commandes . . . 3
4.3 Interprétation requise . . . 3

5 Phase P2 — Observation outillée **3**
5.1 Pipeline . . . 3
5.2 Commandes et preuves . . . 3

6 Phase P3 — Programmation compositionnelle et compilation réelle **3**
6.1 Chaîne de compilation . . . 3
6.2 Cas complexe construit . . . 3
6.3 Commandes . . . 3

7 Phase P4 — Récompense/punition et causalité **4**

8 Phase P5 — Intégration du chat **4**

9 Phase P6 — Campagne de tests **4**

10 Phase P7 — Sauvegarde et documentation **4**

11 Phase P8 — Intégrité de la livraison **4**

12 Registre des incidents et interprétation **4**

13 Conséquences et statut **5**
13.1 Exécution ciblée P1 — registre du bundle local Transformers . . . 5
13.2 Tentative d'installation du runtime Transformers . . . 5
13.3 Incident P2 — garde hérité bloquant le runtime descendant . . . 6
13.4 Incident P2 — contrat de lecture mémoire dans le test . . . 6
13.5 Retest P2 — observation image/PDF dans le runtime public . . . 7
13.6 Incident P2 — preuve vidéo incomplète malgré statut de commande réussi . . . 7
13.7 Retest technique vidéo — marge temporelle insuffisante . . . 8
13.8 Incident P2 — variable de confidentialité non définie . . . 8
13.9 Résultat de retest P2 — confidentialité et échantillonnage vidéo . . . 9

14 Réconciliation finale de la campagne P6 et démarrage P7 **9**
14.1 Description et architecture 9
14.2 Pipeline et commandes exécutées 9
14.3 Résultats et interprétation 9
14.4 Conséquences : théoriques, conceptuelles et opérationnelles 10
14.5 Conclusion et état 10

15 Phase P7 — documentation finale et sauvegarde restaurable **10**
15.1 Description, architecture et pipeline 10
15.2 Commandes d'exécution 10
15.3 Résultats et interprétation 10
15.4 Conséquences et conclusions 10

1 Description

Ce laboratoire documente les défauts observés dans O94.4, les tests de référence, les corrections O94.5, les échecs et les résultats. Il ne remplace pas la checklist exécutable : `checklist_de_travail_O94_5.json` est la source de séquencement ; les deux registres d'erreurs séparent défaut de modèle et défaut d'environnement/exécution.

2 Architecture et protocole expérimental

Les exécutions sont isolées par répertoire de travail, utilisent des budgets explicites et enregistrent commande, code de retour, sortie, hashes d'entrées et de livrables. Les tests unitaires vérifient les contrats ; les tests d'intégration exécutent les outils du poste ; les tests de bout en bout passent par le runtime réel. Une suite qui dépasse son budget, ou dont le code retour n'est pas recueilli, n'est ni PASS ni FAIL : elle est consignée comme interrompue.

3 Phase P0 — Audit et reproduction du défaut

3.1 Problèmes observés

— La voie Transformers exigeait un dossier mais ne gérait pas convenablement la sélection d'un fichier de poids isolé ; l'inventaire ne calculait pas le SHA-256 des gros poids.

— Le gabarit Python d'adaptateur contenait un point `NotImplementedError` ; il ne chargeait donc pas le modèle demandé.

— La première génération compositionnelle créait un graphe JSON O94.4 mais ne produisait pas de source compilée et de bytecode exécutable.

— L'observation multimodale ne faisait pas de fallback OCR pour un PDF scanné et les sommes de contrôle de rapport pouvaient être ambiguës après réécriture.

— Les préférences utilisateur n'étaient pas séparées d'un retour sur la fiabilité opérationnelle intégrée à toutes les tâches du runtime.

3.2 Test de référence

Commande :

```
pytest -q tests/test_o944_python_adapter.py tests/test_o94_local_llm_adapter.py tests/test_o943_
```

Résultat enregistré : 18 tests réussis en 4,68 s. Cette base ne prouvait pas le chargement effectif de SmolLM2 ni la compilation d'un programme Python issue du DSL O94.5. Les fichiers SmolLM2 ne sont pas dans les archives fournies.

4 Phase P1 — Modèles Transformers et poids locaux

4.1 Architecture

L'inspecteur accepte le dossier ou fichier sélectionné ; vérifie configuration, poids et tokenizer ; calcule les hash de contenu en streaming ; stocke l'inventaire dans le registre ; valide de nouveau l'inventaire avant la première charge. Le chargement passe par le modèle Auto de Transformers avec accès réseau désactivé et code distant interdit. Le chemin .pt/.pth de type state-dict exige un fichier de configuration cohérent et un chargement strict des poids seulement.

4.2 Commandes

```
python -m pip install -r requirements-transformers.txt
pytest -q tests/test_o945_transformers_bundle.py
```

4.3 Interprétation requise

Un test avec bundle factice valide l'inspection/enregistrement et le refus d'une empreinte modifiée, mais pas l'inférence de SmolLM2. Cette dernière exige les vrais poids/config/tokenizer sur la machine. Si la dépendance Transformers n'est pas présente, le runtime doit retourner un message exploitable sans téléchargement silencieux.

5 Phase P2 — Observation outillée

5.1 Pipeline

Le runtime observe un fichier local, appelle le décodeur adapté, conserve la source en lecture seule, calcule son hash, crée une empreinte AAM du texte extrait et peut envoyer une copie dérivée au RAG avec provenance. Pour un PDF sans couche de texte, le fallback local est `pdftoppm` puis Tesseract sur au plus huit pages. L'audio n'est décodé que sur une fenêtre acoustique bornée ; une transcription n'est produite que si un modèle ASR local est réellement présent.

5.2 Commandes et preuves

```
python -m py_compile aam_o93/observation_tools.py aam_o93/runtime.py
pytest -q tests/test_o945_observation_reward.py
```

Les reçus doivent inclure l'extraction OCR réelle, le hash du PDF image-only, le hash du rapport final calculé depuis les octets réellement écrits et l'absence de modification de la source.

6 Phase P3 — Programmation compositionnelle et compilation réelle

6.1 Chaîne de compilation

JSON DSL versionné → validation → émission Python → compilation CPython et bytecode → manifeste de hash → exécution contrôlée. Les composants arbitraires sont refusés ; un composant d'atelier qui lancerait du code projet ne peut pas être appelé par une composition générique.

6.2 Cas complexe construit

Le programme `code_workbench_dispatch_v1` orchestre validation de requête, opération de l'atelier de code et scellement du résultat. L'atelier associé réalise les opérations de projet (lister, ouvrir, sauvegarder atomiquement, détecter conflit, rechercher/remplacer, compiler avec CPython, exécuter sous confirmation et timeout). Cette composition vérifie un workflow d'ingénierie logicielle réel ; elle ne prétend pas synthétiser spontanément un IDE industriel complet avec toutes les fonctionnalités de langages.

6.3 Commandes

```
pytest -q tests/test_o945_composition_compiler_workbench.py
python -m aam_o93.composed_programs.code_workbench_dispatch_v1 < input.json
```

7 Phase P4 — Récompense/punition et causalité

Trois canaux séparés sont testés : note de satisfaction utilisateur, fiabilité d'exécution et vérification indépendante. Ils possèdent des EWMAs distinctes, des bornes, un lien d'événement causal et la déduplication des événements. Les états non concluants ne sont pas automatiquement notés comme une faute. La priorisation des compétences consulte l'ajustement combiné lors d'une sélection automatique.

```
pytest -q tests/test_o945_observation_reward.py tests/test_o94_trace_resolution_reward.py
```

Critères de sortie : un succès operational change le score de fiabilité ; une erreur le fait baisser ; un même hash n'est appliqué qu'une fois ; un résultat vérifié possède le hash des critères/preuves ; une note utilisateur ne modifie pas les champs de validation objective.

8 Phase P5 — Intégration du chat

Le modèle sélectionné par l'utilisateur est prioritaire. En automatique, un modèle intégré disponible peut être choisi selon la date d'enregistrement ; si sa génération échoue, l'erreur remonte et aucun autre modèle n'est substitué silencieusement. Le RAG prépare le contexte en fonction de la requête, la réponse conserve la provenance, et les préférences n'affirment pas la vérité. Un test intégration doit exercer ce chemin, pas seulement appeler l'adaptateur directement.

9 Phase P6 — Campagne de tests

La campagne cible d'abord l'inspecteur modèle, les outils OCR/PDF, les opérations de l'atelier, la compilation et la récompense ; une campagne globale est relancée ensuite. Le résultat exact, les durées et les avertissements sont inscrits dans `RAPPORT_TESTS_O94_5_FR.md` ; aucun échec ou warning ne sera supprimé de la restitution.

10 Phase P7 — Sauvegarde et documentation

Les chemins de travail se trouvent dans `chemins_O94_5.json`. Le point `sav_AAM_O94_5` garde le manifeste parent O94.4 sans le modifier et un overlay distinct O94.5. La sauvegarde est restaurable et exclut les logs volatiles du hash d'exécution, tout en conservant ceux-ci en audit.

11 Phase P8 — Intégrité de la livraison

Les archives `Dmodel_AAM_O94_5.zip`, `Amodel_AAM_O94_5.zip` et `suivi_model_AAM_O94_5.zip` sont reconstruites depuis l'état de travail consolidé. Pour chaque archive : test CRC, vérification du manifeste, extraction dans un dossier vierge, préflight et vérification de taille inférieure à 220 Mio. En cas de dépassement, un script de reconstitution est livré.

12 Registre des incidents et interprétation

Les incidents de code et ceux dus à l'environnement sont conservés séparément. À date du présent laboratoire, l'absence des poids SmolLM2 et la dépendance Transformers non installable dans cette session en raison de l'absence d'accès réseau sont des contraintes de validation, et non la preuve que le chargeur ne fonctionne pas. Les tests de fixtures peuvent établir un contrat ; ils ne sont pas présentés comme un test de production SmolLM2.

13 Conséquences et statut

Théorie : les observations sont traitées comme preuve de provenance et non comme vérité sémantique automatique. Conception : les composants sont distingués des compositions et des vérificateurs. Opérationnel : les demandes produisent des artefacts, statuts et hash permettant diagnostic et restauration. Le statut effectif des phases est celui des reçus et de la checklist, mise à jour après chaque campagne. Aucune conclusion sur le potentiel futur d'AAM n'est inférée d'une contrainte d'environnement.

13.1 Exécution ciblée P1 — registre du bundle local Transformers

Description. Lancement de la batterie ciblée après correction de `pytest.ini` et réparation du chemin de preuve P0.

Architecture et pipeline. Inspecter bundle local, inscrire le dossier parent d'un poids sélectionné, vérifier l'empreinte à l'inscription et détecter l'absence de dépendance native sans aucun téléchargement implicite.

Commande d'exécution. `pytest -q tests/test_o945_transformers_bundle.py --disable-warnings --m`

Résultats. Code retour 0 ; 5 tests réussis en 1,19 s. Reçu `receipts/O94_5/P1_transformers_bundle.json` ; logs dans `logs/O94_5/P1_transformers_bundle.*.txt`.

Interprétation. La sélection du dossier parent, les contrôles d'intégrité, les erreurs propres et le trajet registre→génération sont validés par API simulée. Cela ne prouve pas encore le calcul sur un véritable bundle SmolLM2 : Transformers et les poids SmolLM2 ne sont pas présents dans l'environnement de départ.

Conséquences. Théorique : une sélection d'un fichier de poids ne doit pas permettre d'inférer l'architecture sans `config.json`. Conceptuelle : le dossier du modèle est l'unité d'intégration ; un checkpoint brut reste lié à sa configuration et à son tokenizer. Opérationnelle : installer le backend natif et fournir le bundle complet pour tester les tokens réels.

Conclusion et état. P1 reste `IN_PROGRESS` jusqu'à un test de génération native contrôlé ou jusqu'à une preuve reproductible du blocage de dépendance indiquée par le contrat ; aucun poids absent n'est déclaré testé.

13.2 Tentative d'installation du runtime Transformers

Description. Tentative d'ajouter les dépendances optionnelles nécessaires pour effectuer une inférence native sur un mini-modèle local construit pour le test.

Architecture et pipeline. Installer Transformers, Accelerate et SentencePiece dans l'environnement isolé ; aucune ressource de modèle externe ne serait téléchargée.

Commande d'exécution. `python -m pip install -disable-pip-version-check -no-input 'transformer 'accelerate>=1.2,<2' 'sentencepiece>=0.2,<1'`.

Résultats. Code retour 1 ; DNS/connexion vers l'index de paquets impossible. Reçu `receipts/O94_5/P1_optiona`

Interprétation. Il s'agit d'un blocage de l'environnement, pas d'une preuve sur la capacité ou incapacité future d'AAM. La campagne O94.5 continue avec les dépendances déjà présentes, et le chemin d'inférence non testé demeure signalé.

Conséquences. Théorique : séparer preuve de contrat, preuve d'intégration et preuve de performance sur poids réels. Conceptuelle : le poids seul ne définit pas l'architecture ni le tokenizer. Opérationnelle : sur une machine connectée, installer le fichier
textttrequirements-transformers.txt puis valider le bundle SmolLM2 complet.

Conclusion et état. P1 est clôturée selon ses critères prédéfinis de bundle, registre, dispatch simulé et erreur propre ; la validation avec poids SmolLM2 réels reste ouverte comme incident de couverture, non comme succès supposé.

13.3 Incident P2 — garde hérité bloquant le runtime descendant

Description. Le chemin public `AAMRuntime` a échoué avant l'observation OCR parce qu'il consultait d'abord les hashes immuables des versions précédentes, qui sont différents des sources O94.5.

Architecture et pipeline. Constructeur runtime → préflight central → garde O94.3/O94.4 → arrêt avant appel `observe_media`.

Commande d'exécution. `pytest -q tests/test_o945_observation_reward.py -disable-warnings -maxfail=2`.

Résultats. 2 tests passés et 1 échec ; reçu `receipts/O94_5/P2_observation_reward_initial.json`. L'échec est une erreur d'intégration de release, pas un échec Tesseract.

Interprétation. Un descendant ne peut pas être validé en traitant ses changements contrôlés comme s'ils devaient avoir les hashes de ses parents. Le parent doit rester figé ; le descendant doit être validé par son overlay.

Conséquences. Théorique : le contrat d'intégrité suit le graphe de versions. Conceptuelle : le préflight doit router vers le gate de la release active. Opérationnel : modifier `preflight.py`, puis répéter le même test via le chemin public.

Conclusion et état. Incident `O945-M005` ouvert avant correction ; P2 ne peut pas être clôturée sur ce résultat.

13.4 Incident P2 — contrat de lecture mémoire dans le test

Description. Après le routage du préflight O94.5, l'observation publique a atteint Tesseract, le calcul de l'empreinte, l'ingestion RAG et la vérification cryptographique du rapport ; le test a ensuite invoqué une méthode mémoire inexistante.

Architecture et pipeline. `AAMRuntime.observe_media` → Tesseract → fingerprint → RAG → rapport vérifié → assertion de journal mémoire.

Commande d'exécution. `pytest -q tests/test_o945_observation_reward.py -disable-warnings -maxfail=2`.

Résultats. 2 tests réussis, 1 test échoue à `O93Memory.read_events` ; le contrat expose `recent_events`. Reçu `receipts/O94_5/P2_observation_reward_retest.json`.

Interprétation. Le résultat n'indique pas un échec OCR. Il indique que l'assertion de traçabilité doit cibler l'API mémoire réellement implémentée puis contrôler le contenu retourné.

Conséquences. Théorique : la traçabilité doit relier observation et mémoire sans supposer une API. Conceptuelle : test et contrat doivent s'accorder. Opérationnel : examiner `recent_events`, adapter l'assertion et rejouer le même parcours.

Conclusion et état. Incident de harnais `O945-ENV-005` ouvert avant retest ; phase P2 reste active.

13.5 Retest P2 — observation image/PDF dans le runtime public

Description. Rejeu après routage du préflight vers le gate O94.5 et correction de l'assertion mémoire.

Architecture et pipeline. Entrée image réelle → Tesseract OCR → empreinte via l'observateur AAM → ingestion RAG → rapport immuable et sidecar SHA-256 → événement d'audit mémoire.

Commande d'exécution. `pytest -q tests/test_o945_observation_reward.py -disable-warnings -maxfail=2`.

Résultats. 3 tests réussis en 1,61 s ; le test inclut une image OCR réelle, un PDF image-only rasterisé par Poppler puis OCRisé par Tesseract, et le parcours public du runtime. Reçu `receipts/O94_5/P2_observation_r`

Interprétation. Les outils du système sont réellement exécutés. Le texte extrait est une observation instrumentée, non une preuve que le modèle comprend ou reproduit n'importe quel contenu. Audio/vidéo doivent avoir leurs propres tests décodés.

Conséquences. Théorique : séparer décodage, empreinte, représentation RAG et interprétation. Conceptuelle : conserver provenance, hashes et statut de chaque étape. Opérationnel : étendre la campagne à WAV et MP4 locaux.

Conclusion et état. Le bloqueur préflight et l'assertion mémoire sont résolus ; P2 reste `IN_PROGRESS` jusqu'aux tests audio/vidéo et processus.

13.6 Incident P2 — preuve vidéo incomplète malgré statut de commande réussi

Description. Le test crée une vraie vidéo de mouvement avec FFmpeg puis lance l'extracteur O94.5. Le processus FFmpeg retourne zéro pour les images, mais le rapport ne contient pas de hash d'au moins une frame.

Architecture et pipeline. GIF animé généré localement → encodeur FFmpeg → FFprobe → échantillonnage par timestamp → PNG de preuve → SHA-256 → OCR facultatif.

Commande d'exécution. `pytest -q tests/test_o945_observation_reward.py -disable-warnings -maxfail=2`.

Résultats. 3 tests réussis, 1 échec à l'assertion de hash frame ; reçu `receipts/O94_5/P2_audio_video.json`.

Interprétation. Un code retour nul sans vérification du fichier de sortie est insuffisant pour établir qu'une image a été extraite. Ce constat est traité comme un défaut réel de l'outil d'observation jusqu'à preuve contraire.

Conséquences. Théorique : l'observation doit relier l'événement de décodage à un artefact matérialisé et hashé. Conceptuelle : distinguer commande réussie, frame produite et frame analysée. Opérationnel : capturer les erreurs FFmpeg, contrôler l'existence/taille du PNG et ajouter un test de régression.

Conclusion et état. Incident `O945-M006` ouvert ; P2 ne peut pas être clôturée.

13.7 Retest technique vidéo — marge temporelle insuffisante

Description. La première correction a empêché les faux positifs, mais le dernier PNG reste absent avec un seek d'entrée sur une vidéo de seulement une seconde.

Architecture et pipeline. FFmpeg positionne le décodeur en entrée à un timestamp très proche de la durée ; selon les timestamps/points de recherche, aucune frame ne peut être émise même avec un code retour nul.

Commande d'exécution. `pytest -q tests/test_o945_observation_reward.py -disable-warnings -maxfail=2`.

Résultats. 3 tests réussis et le scénario audio/vidéo échoue sur une frame marquée explicitement `OUTPUT_MISSING` ; reçu `receipts/O94_5/P2_video_frame_fix.json`.

Interprétation. La correction d'intégrité a rendu le diagnostic visible ; le problème de positionnement du décodeur reste à corriger en déplaçant `-ss` après l'entrée.

Conséquences. Théorique : code de sortie et existence d'artefact sont deux preuves différentes. Conceptuelle : `-ss` en entrée est un seek rapide mais moins précis ; un seek de sortie est pertinent pour les petites vidéos de test. Opérationnel : décoder depuis l'entrée et appliquer le seek de sortie, sous budget de temps.

Conclusion et état. `O945-M006` reste ouvert jusqu'à ce qu'un nouvel essai prouve deux frames réellement extraites.

13.8 Incident P2 — variable de confidentialité non définie

Description. Après les changements de vérification vidéo, l'observateur a levé `NameError` avant d'inspecter l'image : le champ `persist_text` n'existe pas dans la signature `observe_media`.

Architecture et pipeline. `observe_media` → construction de la politique de confidentialité → `NameError` avant décodage.

Commande d'exécution. `pytest -q tests/test_o945_observation_reward.py -disable-warnings -maxfail=2`.

Résultats. 1 test réussi, 2 tests échoués sous `-maxfail=2` ; reçu `receipts/O94_5/P2_video_output_seek.json`. Le second échec est un gate savepoint non rafraîchi après édition et n'est pas compté comme échec OCR.

Interprétation. C'est un défaut réel de code pour la variable, et une déviation de protocole pour le second échec. Les deux sont conservés dans des registres distincts.

Conséquences. Théorique : les politiques de confidentialité doivent être explicites et testables. Conceptuelle : ne pas confondre le texte rendu au client et le texte persisté dans le rapport. Opérationnel : relier l'option déclarée à une variable définie, rafraîchir la sauvegarde, puis refaire l'essai entier.

Conclusion et état. Incidents `O945-M007` et `O945-ENV-006` ouverts jusqu'au retest.

13.9 Résultat de retest P2 — confidentialité et échantillonnage vidéo

Description. Après correction de la variable de politique texte et refresh du point de sauvegarde, le test image/PDF direct passe, tandis que l'intégration runtime a rencontré un mismatch de snapshot lors de sa construction ; le test vidéo continue de détecter une frame absente.

Architecture et pipeline. Outil direct image/PDF → décodage réussi ; runtime → gate descendant → actuellement à revérifier ; vidéo → timestamps FFprobe → extraction FFmpeg vérifiée par fichier.

Commande d'exécution. `pytest -q tests/test_o945_observation_reward.py -disable-warnings -maxfail=2`.

Résultats. 2 tests passés et 2 échoués ; reçu `receipts/O94_5/P2_observation_privacy_fix.json`. Le défaut NameError est résolu ; les deux autres points restent ouverts.

Interprétation. La surveillance explicite de l'existence des fichiers évite le faux `PASS`. La distribution des timestamps doit exclure la zone finale du flux vidéo qui n'émet pas systématiquement de frame.

Conséquences. Théorique : une preuve instrumentale exige l'artefact. Conceptuelle : les statuts par étape doivent différencier source, decode, frame, OCR. Opérationnel : échantillonner les intervalles internes (par divisions de durée), puis valider les hash.

Conclusion et état. `O945-M006` reste ouvert ; `O945-M007` est résolu sur le test direct image/PDF. La validation du runtime public et la vidéo/audio demandent un nouveau passage.

14 Réconciliation finale de la campagne P6 et démarrage P7

14.1 Description et architecture

Après correction du statut attendu par deux tests média, la suite complète a été rejouée avec une fenêtre Xvfb neuve. Le runner lit les quatre fichiers de contrôle de version, valide le parent O94.4 figé et l'overlay O94.5, écrit un reçu et un journal uniques par tentative, puis met à jour les alias seulement à la sortie de pytest. Ce changement évite qu'un journal mutable modifie le manifeste au milieu d'une construction du runtime.

14.2 Pipeline et commandes exécutées

```
python scripts/refresh_o945_savepoint.py
python scripts/preflight_o945.py
python scripts/run_o945_full_suite.py --timeout-seconds 420
```

Le runner a exécuté `python -X faulthandler -m pytest -q tests -disable-warnings -tb=short` sous Xvfb. Le reçu est `receipts/O94_5/P6_full_suite_regression_20261020T114952Z_19680.json` ; le journal, lié à son hash, est `logs/O94_5/P6_full_suite_regression_20261020T114952Z_19680.log`.

14.3 Résultats et interprétation

Résultat final : **419 passés, 0 échecs, 5 avertissements et 21 sous-tests** en 199,38 s. Le statut du reçu est `PASS_BOUNDED`, le code retour 0 et `timed_out=false`. Un essai précédent avait affiché un résumé complet mais le canal d'orchestration s'était interrompu avant le reçu ; il a été inscrit comme incident de harnais, puis le runner détaché a produit la preuve finale complète. Les deux tests média en défaut ont été rejoués séparément et passent désormais 2/2. Aucune assertion d'intégrité ou d'extraction n'a été retirée.

Les preuves ciblées complémentaires sont : observation outillée 7/0, composition/compilateur/éditeur/chat 10/0, contrats modèles locaux 15/0, reward/autonomie 11/0, observation live/CLI 8/0, RAG/mémoire/échantillon SMS 34/0, contrat de statut 2/0. Ces comptes ciblés recouvrent une partie des tests de la suite complète ; ils ne s'additionnent pas aux 419.

14.4 Conséquences : théoriques, conceptuelles et opérationnelles

Théorique : l'outil d'observation ne transforme pas une extraction en preuve de compréhension ; le hash source reste distinct de l'empreinte AAM et du résultat d'un vérificateur. Conceptuelle : l'éditeur, le compilateur DSL, le runtime composé et le vérificateur ont des rôles distincts et interconnectés. Opérationnelle : la suite reproductible, les reçus, les logs, la matrice de preuve et la sauvegarde permettent un audit indépendant.

14.5 Conclusion et état

P6 est clôturée avec le statut `COMPLETE_EVIDENCED`. P7 devient la phase active pour finaliser la documentation, la restauration et le rapprochement des registres. P8 ne commence qu'après vérification du formalisme, du laboratoire, des instructions CLI/GUI, de la restauration du point de sauvegarde et des dépendances/documents livrables. Le vrai bundle SmolLM2 n'étant pas présent dans cette session, son inférence est explicitement non certifiée ; la procédure de qualification est donnée dans le guide et l'incident demeure ouvert.

15 Phase P7 — documentation finale et sauvegarde restaurable

15.1 Description, architecture et pipeline

P7 ajoute un préflight O94.5 autonome au lieu de réutiliser le préflight historique O94.4, un outil de vérification/restauration des snapshots, et un audit croisé des références entre documents, matrices, registre de preuves et reçus. Le point de sauvegarde garde O94.4 dans un parent immuable et enregistre les deltas O94.5 dans un manifeste séparé.

15.2 Commandes d'exécution

```
python scripts/refresh_o945_savepoint.py
python scripts/preflight_o945.py
python -m pytest -q tests/test_o945_savepoint_restore.py tests/test_o945_standalone_preflight.py
python scripts/restore_o945_savepoint.py
python scripts/audit_o945_documentation.py
```

15.3 Résultats et interprétation

Le reçu de clôture visé pour cette étape est `receipts/O94_5/P7_documentation_restore_final.json`. Les tests couvrent une restauration explicite sur un projet temporaire, la préservation des modifications avant remplacement, le rejet de chemins de traversée et la corruption du snapshot ; le préflight et l'audit de références sont exécutés sur la racine de livraison. Les journaux intermédiaires échoués restent présents et sont liés au registre d'incidents ; seul le reçu final vert sert à la clôture.

15.4 Conséquences et conclusions

Théorique : un manifeste ne prouve pas la justesse d'un algorithme, mais une restauration reproductible permet de retrouver un état connu. Conceptuelle : les reçus de phase, les artefacts et les journaux ont des rôles distincts. Opérationnelle : P7 ne peut se clore qu'après un préflight O94.5 vert, une restauration vérifiée, un audit sans référence cassée et la compilation effective des deux sources LaTeX. P8 reste fermée tant que ces critères n'ont pas été satisfaits.